

Lecture 5: Neural Architectures for 3D Data (Part I)

Equivariant and Modern Latent Representations

Xiangru Huang

WestLake University

2026-03-28

- ① Equivariant neural networks (Leonidas Guibas line, Yaron Lipman line)
- ② 3DShape2Vec lineage
 - 2.1 Structured latent: NSVF, Local Implicit Grid
 - 2.2 Set latent: 3DShape2VecSet, LION
 - 2.3 Discrete latent: VQ/tokenization, ShapeFormer, MeshGPT
 - 2.4 Multi-modal latent: Michelangelo

Why a Part II?

- Part I focused on representation families (point/voxel/mesh/implicit) and classic architectures.
- Part II focuses on **symmetry-aware learning** and **modern latent design**.
- The key shift: from hand-crafted geometric pipelines to **inductive bias + learned latent geometry**.

Roadmap and guiding questions

- Q1: How do we enforce physical/geometric symmetry (rotation, translation)?
- Q2: How should a latent code be organized: grid, set, tokens, or multi-modal?
- Q3: What are the concrete data flows and objectives in modern 3D generative systems?

Symmetry as an architectural prior

Symmetry preliminaries (group action)

- A transformation group G (e.g., rotations) acts on input space: $x \mapsto g \cdot x$.
- We want model behavior consistent with this action.
- In 3D, common groups: $SO(3)$, $SE(3)$, reflections.

Equivariance vs invariance

$$f(g \cdot x) = g \cdot f(x) \quad (\text{equivariance})$$

$$f(g \cdot x) = f(x) \quad (\text{invariance})$$

- Detection/segmentation often needs equivariance in intermediate layers.
- Classification head often uses an invariant readout.

Why equivariance matters in 3D

- Naive data augmentation is sample-inefficient for continuous rotations.
- Equivariant layers reduce orientation burden on optimization.
- Better OOD behavior under unseen poses and sensor coordinate frames.

3D groups in practice

- $SO(3)$: pure rotations around origin.
- $SE(3)$: rigid transforms (rotation + translation).
- Translation equivariance is natural in CNNs; rotation equivariance is harder.

$$SE(3) = SO(3) \ltimes \mathbb{R}^3$$

Representations and irreps (conceptual)

- Equivariant features are typed by how they transform under G .
- Scalar channels: invariant under rotation.
- Vector/tensor channels: rotate with the input.
- Spherical harmonics and irreducible representations (irreps) parameterize this behavior.

Data flow template of an SE(3)-equivariant block

$$\{(\mathbf{p}_i, \mathbf{h}_i)\}_{i=1}^N \xrightarrow{\text{neighborhood}} \mathcal{N}(i) \xrightarrow{\text{equivariant kernel}} \mathbf{h}'_i \xrightarrow{\text{nonlinearity on typed features}} \mathbf{h}_i^+$$

- Inputs: positions $\mathbf{p}_i$ and typed features $\mathbf{h}_i$.
- Message depends on relative geometry $(\mathbf{p}_j - \mathbf{p}_i)$ and feature types.
- Output keeps transformation law by construction.

Tensor Field Networks (Guibas group lineage)

- Uses spherical harmonics and radial kernels to build $SE(3)$ -equivariant messages.
- Features are organized by rotation order (type-0, type-1, ...).
- Strong theoretical equivariance, but heavier implementation and compute.

SE(3)-Transformer (high-level flow)

$(\mathbf{P}, \mathbf{H}) \rightarrow$ equivariant Q/K/V $\rightarrow$ attention weights from geometric relations $\rightarrow \mathbf{H}' \rightarrow$ typed residual stack

- Attention generalized to typed geometric features.
- Good expressivity for molecular and 3D geometric tasks.

EGNN-style design (lightweight SE(3)-equivariance)

- Avoids spherical harmonics; uses distances and coordinate updates.
- Messages are scalar functions of pairwise distances and node states.
- Coordinates are updated in an equivariant way:

$$\mathbf{p}'_i = \mathbf{p}_i + \sum_{j \in \mathcal{N}(i)} (\mathbf{p}_i - \mathbf{p}_j) \phi_{ij}$$

Vector Neurons (Guibas line): idea

- Replace scalar channels by 3D vector channels.
- Linear maps and nonlinearities are redesigned to preserve rotation behavior.
- Practical way to inject $SO(3)$ -equivariance with familiar deep-learning ops.

Yaron Lipman line: key perspective

- Geometry processing + learning: enforce intrinsic consistency and symmetry.
- Frame/canonicalization and equivariant constructions improve stability.
- Theme: build models whose outputs transform correctly, not just empirically robust.

Equivariant model trade-offs

- Strong equivariance often increases implementation complexity.
- Approximate/lightweight methods are easier but may lose strict guarantees.
- Choice depends on task: molecular modeling and physics often benefit strongly.

Practical checklist for equivariant design

- Define the exact group symmetry needed ($SO(3)$, $SE(3)$, reflection?).
- Decide feature types (scalar/vector/tensor channels).
- Keep all operations symmetry-compatible: message, aggregation, nonlinearity, readout.
- Verify numerically: transform input and test output consistency.

Part 1 summary

- Equivariance is an architectural prior, not just augmentation.
- Guibas/Lipman-related lines push symmetry-aware geometric learning.
- Modern 3D systems increasingly mix equivariant blocks with transformers/latent models.

Structured, set, discrete, and multi-modal latents

What is the 3DShape2Vec perspective?

- Learn reusable latent codes for 3D geometry and appearance.
- Decode latents into point/mesh/implicit fields/radiance.
- Build a latent space that supports reconstruction, editing, and generation.

Latent taxonomy

Type	Structure	Typical decoder
Structured	Grid/hierarchy	local implicit / volume rendering
Set	unordered tokens	set transformer / diffusion
Discrete	codebook tokens	autoregressive transformer
Multi-modal	joint text-image-3D	unified latent decoder

NSVF and Local Implicit Grid

Structured latent: core intuition

- Keep a spatial index (grid/octree/hash) in latent form.
- Query-time interpolation gives local features for coordinates.
- Decoder predicts density/color/SDF from local latent + coordinate encoding.

Neural Sparse Voxel Fields (NSVF): concept

- Sparse voxel structure stores latent features only in occupied regions.
- Each ray intersects a small set of active voxels.
- Neural rendering combines sparse acceleration with learned fields.

scene $\rightarrow$ sparse voxel occupancy + latent grid $\rightarrow$ ray marching on active voxels $\rightarrow$ $(\sigma, \mathbf{c})$ queries $\rightarrow$ volume rendering

- Voxel pruning updates occupancy during training.
- Sparse traversal reduces wasted samples in empty space.

NSVF training objective

- Photometric loss between rendered and target pixels.
- Regularizers for sparsity and geometry smoothness.
- Optimization alternates between field parameters and occupancy structure.

Local Implicit Grid: concept

- Store latent vectors on a regular or multiresolution grid.
- For query point $\mathbf{x}$, interpolate nearby grid latents.
- Decode occupancy/SDF/color via local MLP conditioned on interpolated latent.

Local Implicit Grid data flow

$$\mathbf{x} \in \mathbb{R}^3 \xrightarrow{\text{locate cell}} \{\mathbf{z}_{c_1}, \dots, \mathbf{z}_{c_8}\} \xrightarrow{\text{trilinear interp}} \mathbf{z}(\mathbf{x}) \xrightarrow{[\mathbf{z}(\mathbf{x}), \gamma(\mathbf{x})]} \text{MLP} \rightarrow \hat{o}(\mathbf{x}) \text{ or } \hat{s}(\mathbf{x})$$

- Locality gives detail; multiscale grids improve global consistency.

Structured latent: strengths and limits

- Pros: fast queries, strong locality prior, scalable with sparsity.
- Cons: memory tied to spatial discretization; resolution/generalization trade-off.
- Best for scenes/shapes where local continuity dominates.

3DShape2VecSet and LION

Set latent: why unordered tokens?

- Shape parts are not naturally rasterized on a fixed grid.
- Unordered latent set allows permutation-invariant representation.
- Works well with set transformers and diffusion on sets.

3DShape2VecSet: high-level architecture

shape $\rightarrow$ encoder $\rightarrow \{\mathbf{z}_1, \dots, \mathbf{z}_M\} \rightarrow$ set interaction block $\rightarrow$ implicit decoder / point decoder $\rightarrow \hat{\text{shape}}$

- $\{\mathbf{z}_m\}$ are latent slots for geometry components.

3DShape2VecSet data flow details

- Encoder produces token features and optional token positions.
- Cross-token attention exchanges global context.
- Decoder queries each spatial point against set latents:

$$\phi(\mathbf{x}) = \text{Attn}(\mathbf{q}(\mathbf{x}), \{\mathbf{z}_m\}), \quad \hat{o}(\mathbf{x}) = g_{\theta}(\mathbf{x}, \phi(\mathbf{x}))$$

LION: latent set generation idea

- Learn a distribution over latent sets, often via diffusion.
- Sample latent set first, then decode to 3D geometry.
- Separates **shape prior** learning from **renderer/decoder** details.

LION data flow (generator path)

$\epsilon \sim \mathcal{N}(0, I) \rightarrow \text{diffusion denoiser over set tokens} \rightarrow \{\mathbf{z}_1, \dots, \mathbf{z}_M\} \rightarrow \text{3D decoder} \rightarrow \hat{\text{shape}}$

- Training aligns generated token set distribution with encoded real-shape sets.

Set latent: strengths and limits

- Pros: flexible topology, part-aware representation, permutation handling.
- Cons: token-to-geometry assignment is implicit and can be unstable.
- Good for diverse object categories and compositional generation.

VQ, ShapeFormer, MeshGPT

Why discrete tokens in 3D?

- Brings 3D generation closer to language/image token modeling.
- Enables autoregressive transformers and scalable sequence pretraining.
- Improves compression and reuse of geometric primitives.

VQ/tokenization mechanics

$$\mathbf{z}_e(\mathbf{x}) \xrightarrow{\text{nearest code in } \mathcal{C}} \mathbf{e}_k, \quad k = \arg \min_j \|\mathbf{z}_e - \mathbf{e}_j\|_2^2$$

- Encoder output is quantized to a codebook entry.
- Decoder reconstructs shape from token IDs or token embeddings.
- Typical losses: reconstruction + commitment + codebook update.

ShapeFormer: core idea

- Convert 3D structure into token sequence with positional scheme.
- Train transformer to model token dependency over shape structure.
- Supports generation and completion via sequence sampling/inpainting.

ShapeFormer data flow

$$\text{shape} \rightarrow \text{tokenizer} \rightarrow (t_1, \dots, t_T) \xrightarrow{\text{Transformer}} p(t_i \mid t_{<i}) \xrightarrow{\text{sampling}} \hat{t}_{1:T} \xrightarrow{\text{detokenizer}} \hat{\text{shape}}$$

- Structure-aware ordering is critical to quality.

MeshGPT: core idea

- Tokenize mesh components (e.g., vertices/faces/patches) into discrete symbols.
- Use GPT-style autoregressive transformer over mesh token stream.
- Learn long-range mesh regularities directly from token sequences.

mesh $\rightarrow$ mesh tokenizer/VQ $\rightarrow (m_1, \dots, m_T) \rightarrow$ GPT $\rightarrow \hat{m}_{1:T} \rightarrow$ mesh decoder $\rightarrow \hat{\mathcal{M}}$

- Autoregressive decoding supports conditional generation and editing.

Discrete latent: strengths and limits

- Pros: sequence modeling ecosystem, compression, controllable sampling.
- Cons: tokenizer bottleneck, sequence length, decoding latency.
- Quality depends heavily on tokenization granularity and ordering.

Michelangelo and joint latent spaces

Michelangelo: high-level goal

- Build a unified latent space across text, image, and 3D shape signals.
- Support text-to-3D, image-to-3D, and cross-modal editing in one framework.
- Bridge 2D/3D foundation modeling via shared representations.

Michelangelo data flow (conceptual)

text/image/partial-3D $\rightarrow$ modality encoders $\rightarrow \mathbf{z}_{\text{joint}}$ $\rightarrow$ 3D decoder (implicit or mesh) $\rightarrow$ 3D $\hat{\text{asset}}$

- Training combines reconstruction and cross-modal alignment losses.
- Contrastive and generative objectives are often mixed.

Cross-modal objectives and challenges

- Alignment: semantically similar text/image/3D should map nearby in latent space.
- Geometry fidelity: latent alignment must not destroy metric/shape accuracy.
- Data challenge: paired text-image-3D corpora are expensive and noisy.

Takeaways and key references

- Equivariant line: TFN, SE(3)-Transformer, EGNN, Vector Neurons; works associated with Leonidas Guibas and Yaron Lipman lines emphasize symmetry-consistent learning.
- Structured latent: NSVF, Local Implicit Grid.
- Set latent: 3DShape2VecSet, LION.
- Discrete latent: VQ/tokenization, ShapeFormer, MeshGPT.
- Multi-modal latent: Michelangelo.
- Practical trend: hybridize symmetry priors with scalable latent/token pipelines.